

# LLM-Based Architecture Detection in Software Repositories: An Empirical Multi-Model Evaluation

Anonymous Author(s)

Anonymous Institution(s)

**Abstract.** Identifying the dominant architectural style of a software repository is a prerequisite for architecture-aware quality improvement, yet manual classification requires significant expertise and does not scale. This paper investigates whether Large Language Models (LLMs) can infer architectural styles from repository-level structural evidence. We evaluate five LLMs—Qwen3-coder-next, DeepSeek-v3.2, Gemini-3-Flash-Preview, Gemma4-31B, and MiniMax-M2.7—across four incremental context configurations: (1) file tree only, (2) file tree with import dependency graph, (3) adding Python code signatures, and (4) adding repository structural metrics. Applied to 57 manually labeled open-source Python backend repositories spanning three architectural styles (modular monolith, layered, script-based), the best configuration—Qwen3 with file tree and import graph—achieves 70.2% accuracy and 0.65 macro-F1. Import dependency graphs provide the most significant accuracy improvement across all models (+5.3% to +10.5%), while Python code signatures offer no benefit or degrade performance for all five models. All models systematically confuse modular monolith and layered architectures, and none produce well-calibrated confidence scores. These results establish empirical baselines for LLM-based architecture detection and highlight both the potential and current limitations of automated architectural classification.

**Keywords:** software architecture detection · large language models · architectural classification · repository analysis · empirical software engineering

## 1 Introduction

Software architecture fundamentally shapes a system’s quality attributes, including maintainability, modifiability, and scalability [2] [12]. Identifying the dominant architectural style of a codebase is a prerequisite for architecture-aware quality improvement [10], automated refactoring [11], and technical debt assessment [3]. However, architectural knowledge is frequently implicit—embedded in code structure, dependency patterns, and naming conventions rather than documented explicitly [5].

Manual architecture classification requires significant expertise and does not scale to the thousands of repositories encountered in large organizations or empirical research [4]. Static analysis tools can extract structural metrics but lack the semantic reasoning needed to distinguish, for example, a modular monolith from a layered architecture when both exhibit similar package structures [8]. Recent advances in Large Language Models (LLMs) have demonstrated strong capabilities in code understanding [9] [13], including refactoring opportunity detection [11] [6] and code-level quality improvement [17]. However, existing work operates predominantly at the method or class level [6], and no systematic evaluation exists of LLMs’ ability to perform repository-level architectural classification.

We define the **Architecture Detection Problem** as: given repository-level evidence (file structure, dependency graph, code signatures, and/or structural metrics), classify the repository into one of a predefined set of architectural styles. This problem is distinct from architecture recovery [16], which aims to reconstruct a detailed architectural model; our task requires coarse-grained style classification suitable for downstream automation such as tactic selection.

This paper presents the first systematic empirical evaluation of LLM-based architecture detection across multiple models and evidence configurations. We investigate:

- **RQ1:** How accurately can LLMs classify the architectural style of a Python backend repository?
- **RQ2:** Which types of repository-level evidence contribute most to classification accuracy?
- **RQ3:** How do different LLMs compare in architectural classification performance?

Our contributions are: (1) an empirical evaluation framework for LLM-based architecture detection, including a manually labeled dataset of 57 Python repositories across three architectural styles; (2) a systematic comparison of four incremental context configurations across five LLMs, demonstrating that import dependency graphs are the most valuable evidence type; and (3) evidence that all evaluated models systematically confuse modular monolith and layered architectures, revealing a fundamental limitation in current LLM architectural reasoning.

The remainder of this paper is organized as follows: Section 2 reviews background and related work; Section 3 describes the methodology; Section 4 presents results; Section 5 provides discussion; Section 6 addresses threats to validity; and Section 7 concludes.

## 2 Background and Related Work

### 2.1 Software Architecture Classification

Software architecture is defined by its elements, form, and rationale [12], or as a collection of components and connectors in a specific configuration [5]. Ar-

chitectural styles—such as layered, modular monolith, microservices, and event-driven—provide high-level structural templates that constrain how components interact [2].

Architectural decisions directly influence quality attributes. The ISO/IEC 25010 standard [1] identifies modularity, analysability, and modifiability as key maintainability sub-characteristics, all shaped by architectural choices. Márquez et al. [10] systematically mapped architectural tactics across quality attributes, demonstrating the tight coupling between architecture and quality. Rahmati [15] showed that architectural pattern effectiveness is context-dependent, with misapplication increasing maintenance effort by up to 10%. Bogner et al. [3] further demonstrated that modifiability tactics—particularly coupling reduction—are systematically addressed through service-oriented design patterns.

Architecture recovery and detection have been studied through static analysis [8], dynamic analysis, and manual expert assessment [16]. However, automated approaches typically extract structural metrics without performing the semantic reasoning needed for style classification. Kim et al. [7] demonstrated that quality-driven architecture development requires understanding architectural style to select appropriate tactics—motivating our work on automated style detection as an enabling step.

## 2.2 LLMs in Software Engineering

LLMs have demonstrated capabilities across multiple software engineering tasks. In code refactoring, multi-agent frameworks such as MANTRA achieve 82.8% compilable refactoring success [17], and LLM-assisted approaches can improve code quality at the file level. Horikawa et al. [6] found that agentic refactorings are dominated by low-level edits, with agents performing fewer high-level design changes than human developers—a limitation directly relevant to architectural reasoning.

For code understanding, Piao et al. [13] showed that structured prompts with architectural context significantly improve LLM output quality. Liu et al. [9] found that LLMs can identify up to 86.7% of refactoring opportunities but have limited performance in complex synthesis. Shokri and Khatchadourian [16] reported 95% syntactic but only 5% semantic correctness for inter-procedural program synthesis, highlighting the difficulty of cross-module reasoning.

Esposito et al. [4] surveyed generative AI applications for software architecture, identifying architecture detection and classification as a key open area. Pucho et al. [14] demonstrated LLM-based refactoring for Python ML projects, though limited to code-level patterns without architectural reasoning.

## 2.3 Research Gap

While LLMs have been applied to code-level refactoring [11] [14] [17] and architecture-level tactic detection [16], no existing work systematically evaluates LLMs’ ability to classify repository-level architectural styles. Esposito et al. [4] explicitly identified this as a key open area. Our work addresses this gap by providing

the first controlled multi-model evaluation of LLM-based architecture detection across multiple evidence configurations.

### 3 Methodology

#### 3.1 Research Design

This study employs a controlled empirical evaluation comparing five LLMs across four context configurations on a manually labeled dataset. The independent variables are: (1) the LLM model and (2) the evidence provided in the prompt. The dependent variable is classification accuracy against human-assigned ground truth labels. Following the principles of evidence-based software engineering [2], we report standard classification metrics and analyze error patterns to characterize model behavior.

#### 3.2 Architectural Taxonomy

We define three architectural styles applicable to Python backend repositories:

- **Script-based:** Repository centered around few script files with weak package structure and limited modular decomposition. Characterized by flat directory structure, few internal dependencies, and high per-file complexity.
- **Layered:** Repository with clear separation of concerns into architectural layers (e.g., API/controllers, services, repositories/data access, models, configuration). Internal dependencies flow predominantly in one direction between layers.
- **Modular monolith:** Single deployable application organized into multiple packages/modules with meaningful internal boundaries. Distinguished from layered by the absence of strict layer hierarchy—modules may represent features or domains rather than technical layers.

The distinction between modular monolith and layered is maintained despite their conceptual proximity, as they imply different decomposition strategies (feature-based vs. layer-based) with distinct implications for downstream tasks such as architectural tactic selection [2] [10].

#### 3.3 Dataset

Repositories were sourced from public Python backend projects on GitHub via the GitHub Search API. A multi-stage filtering process was applied following established practices for curating research-quality repository datasets [15]. Inclusion criteria required:

- Presence of Python source files (`.py`) and a valid `requirements.txt`;
- Exclusion of ML-heavy projects identified by the presence of Jupyter Notebooks (`.ipynb`) or dominant use of data science libraries;
- Evidence of active development through commit history and star counts (ranging from 10 to 28,230).

After filtering, 59 repositories were retained. One repository failed artifact extraction due to its size, leaving **57 repositories** with complete results across all models and prompts. Each repository was cloned at a fixed commit hash to ensure reproducibility.

Ground truth labels were assigned by a single expert annotator following the taxonomy defined in Section 3.2. Table 1 shows the class distribution.

**Table 1.** Dataset class distribution (N = 57).

| Architecture Style | Count     | Proportion |
|--------------------|-----------|------------|
| Modular monolith   | 33        | 57.9%      |
| Layered            | 16        | 28.1%      |
| Script-based       | 8         | 14.0%      |
| <b>Total</b>       | <b>57</b> |            |

### 3.4 Repository Evidence Extraction

For each repository, we extract three types of structural evidence through automated analysis:

**File tree.** An ASCII representation of the repository directory structure, excluding common non-source directories.

**Import dependency graph.** Internal import relationships between Python modules, extracted via AST parsing. Only internal dependencies are included.

**Python code signatures.** For each .py file, we extract import statements, class definitions with base classes, and function/method signatures using AST parsing. Full function bodies are excluded to reduce context size while preserving structural information.

Additionally, we compute **structural metrics** from the codebase: module count, average fan-out, maximum fan-in, dependency cycle detection, and total dependency edge count.

### 3.5 Prompt Design

We define four incremental context configurations:

- **P1:** File tree only
- **P2:** File tree + import dependency graph
- **P3:** File tree + import dependency graph + Python code signatures
- **P4:** File tree + import dependency graph + Python code signatures + structural metrics

All prompts share an identical system prompt specifying the task, architectural taxonomy, allowed labels, and required JSON output schema.

### 3.6 Models

We evaluate five LLMs, all accessed through the Ollama API with identical parameters (temperature = 0.2, timeout = 300s, retries = 2):

- **Qwen3-coder-next:cloud**
- **DeepSeek-v3.2:cloud**
- **Gemini-3-Flash-Preview:cloud**
- **Gemma4:31B-cloud**
- **MiniMax-M2.7:cloud**

Each prompt is executed exactly once per repository per model.

### 3.7 Evaluation Metrics

We report:

- **Accuracy**
- **Precision, Recall, F1-score** (macro and weighted)
- **Confusion matrices**
- **Confidence analysis**

## 4 Results

### 4.1 Overall Classification Performance (RQ1)

Table 2 presents accuracy and macro-F1 for all model–prompt combinations.

**Table 2.** Accuracy and macro-F1 across models and prompts. Best result per column in bold.

| <b>Model</b> | <b>P1</b>   |                 | <b>P2</b>   |                 | <b>P3</b>   |                 | <b>P4</b>   |                 |
|--------------|-------------|-----------------|-------------|-----------------|-------------|-----------------|-------------|-----------------|
|              | Acc         | F1 <sub>m</sub> | Acc         | F1 <sub>m</sub> | Acc         | F1 <sub>m</sub> | Acc         | F1 <sub>m</sub> |
| Qwen3        | <b>.596</b> | .53             | <b>.702</b> | <b>.65</b>      | .632        | .52             | <b>.667</b> | <b>.57</b>      |
| MiniMax      | .579        | <b>.55</b>      | .649        | .61             | <b>.649</b> | <b>.55</b>      | .649        | .56             |
| Gemma4       | .561        | .48             | .632        | .55             | .614        | .47             | .614        | .45             |
| DeepSeek     | .544        | .46             | .614        | .50             | .614        | .48             | .632        | .49             |
| Gemini       | .526        | .47             | .579        | .49             | .561        | .41             | .596        | .45             |

The best overall result is achieved by **Qwen3 with P2** (file tree + import graph): **70.2% accuracy, 0.65 macro-F1**.

### 4.2 Impact of Evidence Types (RQ2)

Import dependency graphs are the most valuable evidence type, improving accuracy for all five models. Python code signatures provide no benefit or actively harm performance. Structural metrics provide only marginal improvement.

**Table 3.** Accuracy change (%) by evidence type addition.

| Model    | P1→P2         | P2→P3 | P3→P4 |
|----------|---------------|-------|-------|
| Qwen3    | + <b>10.5</b> | −7.0  | +3.5  |
| MiniMax  | +7.0          | 0.0   | 0.0   |
| Gemma4   | +7.0          | −1.8  | 0.0   |
| DeepSeek | +7.0          | 0.0   | +1.8  |
| Gemini   | +5.3          | −1.8  | +3.5  |

**Table 4.** Per-class accuracy at P2 (file tree + import graph).

| Class                 | Qwen3       | MiniMax | Gemma4 | DeepSeek    | Gemini |
|-----------------------|-------------|---------|--------|-------------|--------|
| Modular monolith (33) | <b>.727</b> | .636    | .606   | .515        | .424   |
| Layered (16)          | .750        | .688    | .750   | .750        | .750   |
| Script-based (8)      | .750        | .750    | .750   | <b>.875</b> | .750   |

### 4.3 Per-Class Performance

The primary differentiator between models is modular monolith detection, where Qwen3 substantially outperforms the others.

### 4.4 Model Comparison (RQ3)

**Table 5.** Model profiles across key dimensions.

| Dimension          | Qwen3         | MiniMax | Gemma4 | DeepSeek | Gemini |
|--------------------|---------------|---------|--------|----------|--------|
| Best accuracy      | <b>.702</b>   | .649    | .632   | .632     | .596   |
| Best $F1_m$        | <b>.65</b>    | .61     | .55    | .50      | .49    |
| Import $\Delta$    | + <b>10.5</b> | +7.0    | +7.0   | +7.0     | +5.3   |
| Signature $\Delta$ | −7.0          | 0.0     | −1.8   | 0.0      | −1.8   |
| Mean confidence    | .887          | .826    | .906   | .847     | .913   |

Qwen3 achieves the highest overall accuracy. MiniMax ranks second and shows the best confidence calibration. DeepSeek excels at script-based detection. Gemini shows the lowest overall accuracy but the highest prediction confidence.

### 4.5 Systematic Misclassification Patterns

The dominant error across all models is confusion between modular monolith and layered architectures.

**Table 6.** Modular monolith misclassifications at P2 (N = 33).

| Model    | Correct   | →layered | →script | →other |
|----------|-----------|----------|---------|--------|
| Qwen3    | <b>24</b> | 8        | 1       | 0      |
| MiniMax  | 21        | 7        | 2       | 3      |
| Gemma4   | 20        | 9        | 3       | 1      |
| DeepSeek | 17        | 11       | 5       | 0      |
| Gemini   | 14        | 14       | 4       | 1      |

**Table 7.** Mean confidence: correct vs. incorrect predictions (P2).

| Model    | Correct | Incorrect | $\Delta$    |
|----------|---------|-----------|-------------|
| MiniMax  | .838    | .812      | <b>.026</b> |
| Qwen3    | .893    | .876      | .017        |
| Gemma4   | .918    | .904      | .014        |
| DeepSeek | .853    | .839      | .014        |
| Gemini   | .916    | .907      | .009        |

#### 4.6 Confidence Calibration

No model produces confidence scores that reliably distinguish correct from incorrect predictions.

## 5 Discussion

### 5.1 The Value of Import Graphs

The consistent accuracy improvement from import dependency graphs across all five models is the strongest finding of this study. Import graphs encode inter-module dependency structure, directly reflecting architectural decomposition decisions, while file trees capture only static organizational hierarchy. This aligns with architectural theory, where dependencies between components are a defining characteristic of architectural style [12] [5] [2].

### 5.2 Why Signatures Do Not Help

The finding that Python code signatures degrade or do not improve performance for all five models is robust. Signatures appear to introduce redundancy, misleading naming signals, and context dilution without adding sufficient architectural information.

### 5.3 The Modular Monolith Problem

The systematic confusion between modular monolith and layered architectures is the most significant limitation identified. This confusion is shared across all five models and all context configurations, suggesting a fundamental challenge in both the task and the taxonomy boundary.



## 5.4 Practical Implications

For practical architecture detection:

- file tree + import graph is the best default configuration;
- code signatures should be omitted;
- confidence scores should not be used as a reliability filter;
- modular monolith vs. layered predictions should be treated with caution.

## 6 Threats to Validity

**Internal validity.** Ground truth labels were assigned by a single annotator. Each prompt was executed once per model.

**External validity.** Results are limited to 57 Python backend repositories from GitHub and three architectural styles.

**Construct validity.** Architectural style is a high-level abstraction that may not be fully captured by a single label. Some repositories may exhibit hybrid architectural characteristics.

**Conclusion validity.** The dataset is small and class-imbalanced. Results should be interpreted as preliminary empirical evidence establishing baselines rather than definitive conclusions.

## 7 Conclusion

This study provides the first systematic evaluation of LLM-based architectural style detection across multiple models and evidence configurations.

**RQ1:** LLMs can classify architectural styles with moderate accuracy. The best configuration (Qwen3 with file tree + import graph) achieves 70.2% accuracy and 0.65 macro-F1.

**RQ2:** Import dependency graphs provide the most significant and consistent accuracy improvement across all five models. Python code signatures offer no benefit or actively degrade performance.

**RQ3:** Qwen3 achieves the highest overall accuracy. MiniMax ranks second with the best confidence calibration. DeepSeek excels at script-based detection. All models share a fundamental limitation: systematic confusion between modular monolith and layered architectures.

Future work should expand the dataset, include multiple annotators, investigate few-shot or fine-tuning strategies, and evaluate the downstream impact of architecture detection accuracy on automated architecture-aware transformations.

## Data Availability

An anonymized replication package containing the labeled dataset, extracted repository artifacts (file trees, import graphs, and code signatures), prompt templates, raw model outputs, and evaluation scripts will be made available to the

program committee via an anonymous repository. The final camera-ready version will specify the public archival location of these artifacts.

<https://github.com/anonymous-ghub/Architecture-Detection-in-Software-Repositories>

## References

1. ISO/IEC 25010 — Systems and software engineering: Software product Quality Requirements and Evaluation (SQuaRE) — Product quality model. <https://iso25000.com/en/iso-25000-standards/iso-25010> (2023), [Online; accessed 2025-12-04]
2. Bass, L., Clements, P., Kazman, R.: Software Architecture in Practice. Addison-Wesley, 4th edn. (2021)
3. Bogner, J., Wagner, S., Zimmermann, A.: Using architectural modifiability tactics to examine evolution qualities of service- and microservice-based systems. *Computer Science – Research and Development* **34**, 187–237 (2019). <https://doi.org/10.1007/s00450-019-00402-z>
4. Esposito, M., Li, X., Moreschini, S., Ahmad, N., Cerny, T., Vaidhyanathan, K., Lenarduzzi, V., Taibi, D.: Generative ai for software architecture. applications, trends, challenges, and future directions. *Applications, Trends, Challenges, and Future Directions* (2025)
5. Garlan, D., Shaw, M.: An introduction to software architecture **1**(3) (1993)
6. Horikawa, K., Li, H., Kashiwa, Y., Adams, B., Iida, H., Hassan, A.E.: Agentic refactoring: An empirical study of AI coding agents. In: arXiv preprint arXiv:2511.04824 (2025)
7. Kim, S., Kim, D.K., Lu, L., Park, S.: Quality-driven architecture development using architectural tactics **82**(8), 1211–1231 (2009)
8. Lenarduzzi, V., Pecorelli, F., Saarimaki, N., Lujan, S., Palomba, F.: A critical comparison on six static analysis tools: Detection, agreement, and precision **198**, 111575 (2023)
9. Liu, B., Jiang, Y., Zhang, Y., Niu, N., Li, G., Liu, H.: Exploring the potential of general purpose LLMs in automated software refactoring: An empirical study. *Automated Software Engineering* **32**(1) (2025). <https://doi.org/10.1007/s10515-025-00500-0>
10. Márquez, G., Astudillo, H., Kazman, R.: Architectural tactics in software architecture: A systematic mapping study. *Journal of Systems and Software* **197**, 111558 (2022). <https://doi.org/10.1016/j.jss.2022.111558>
11. Martinez, S., Xu, L., Elnaggar, M., AlOmar, E.A.: Software refactoring research with large language models: A systematic literature review. *Journal of Systems and Software* p. 112762 (2025). <https://doi.org/10.1016/j.jss.2025.112762>
12. Perry, D.E., Wolf, A.L.: Foundations for the study of software architecture **17**(4), 40–52 (1992). <https://doi.org/10.1145/141874.141884>
13. Piao, Y.C.K., Paul, J.C., Da Silva, L., Dakhel, A.M., Hamdaqa, M., Khomh, F.: Refactoring with LLMs: Bridging human expertise and machine understanding. In: arXiv preprint arXiv:2510.03914 (2025)
14. Pucho, A.P., Ferreira, A.M., Cirilo, E.J.R., Cafeo, B.B.: Refactoring python code with llm-based multi-agent systems: An empirical study in ml software projects. In: *Simpósio Brasileiro de Engenharia de Software (SBES)*. pp. 678–684. SBC (2025)
15. Rahmati, Z., Tanhaei, M.: Ensuring software maintainability at software architecture level using architectural patterns **2**(1), 81–102 (2021). <https://doi.org/10.22060/ajmc.2021.19232.1044>

16. Shokri, S.E., Khatchadourian, R.: IPSynth: Interprocedural program synthesis for software architecture recovery and architectural tactics detection. In: arXiv preprint arXiv:2403.10836 (2024)
17. Xu, Y., Lin, F., Yang, J., Chen, T.H., Tsantalis, N.: MANTRA: Enhancing automated method-level refactoring with contextual RAG and multi-agent LLM collaboration. In: arXiv preprint arXiv:2503.14340 (2025)